

Task -1

```
1  #include<iostream>
2  using namespace std;
3  class person{
4      public:
5          string name;
6          int age;
7
8          void displayinfo(){
9              cout<<name<<endl;
10             cout<<age<<endl;
11         }
12     };
13     class Student:public person{
14     public:
15         int roll_Number;
16
17         void displayStudent(){
18
19             cout<<roll_Number<<endl;
20
21         }
22     };
23     int main(){
24         Student s1;
25         s1.name = "Yash";
26         s1.age = 19;
27         s1.roll_Number = 878;
28         s1.displayinfo();
29         s1.displayStudent();
30     }
```

Yash
19
878

Task-2

Petrol
Toyota
120

```
1  #include<iostream>
2  using namespace std;
3  class Vehicle{
4      public:
5
6      string brand;
7      float speed;
8      void showVehicle() {
9          cout<<brand<<endl<<speed<<endl;
10     }
11 };
12 class Car:public Vehicle{
13     public:
14         string fuelType;
15     void showCarDetails(){
16         cout<<fuelType<<endl;
17     }
18 };
19
20 int main(){
21     Car c1;
22     c1.brand = "Toyota";
23     c1.speed = 120;
24     c1.fuelType = "Petrol";
25     c1.showCarDetails();
26     c1.showVehicle();
27 }
```

Task-3

```
1  #include<iostream>
2  using namespace std;
3  class Employee{
4      public:
5          int id;
6          float salary;
7      void showEmployee(){
8          cout<<id<<endl;
9      }
10 };
11 class Manager:public Employee{
12     public:
13         float bonus;
14     void showManager(){
15         cout<<(salary+bonus)<<endl;
16     }
17 };
18 int main(){
19
20     Manager m1;
21     m1.id = 1234;
22     m1.bonus = 12000;
23     m1.salary = 34000;
24     m1.showEmployee();
25     m1.showManager();
26 }
```

1234
46000

Task-4

```
1 #include<iostream>
2 using namespace std;
3 class shape{
4     public:
5         float PI = 3.14;
6         float radius;
7 };
8 class Circle:public shape{
9     public:
10        float area;
11        void Area(){
12            area= PI*radius*radius;
13            cout<<"Area: "<<area<<endl;
14        }
15 };
16 int main(){
17     Circle c1;
18     c1.radius = 12;
19     c1.Area();
20 }
21 }
```

Area: 452.16

Task-5

```
1 #include<iostream>
2 using namespace std;
3 class write{
4     public:
5         void Write(){
6             cout<<"Author writes!"<<endl;
7         }
8 };
9 class speaker{
10     public:
11         void speak(){
12             cout<<"Author speaks!"<<endl;
13         }
14 };
15 class author:public write,public speaker{
16     public:
17         void Author(){
18             cout<<"I am author!"<<endl;
19         }
20 };
21 int main(){
22     author a1;
23     a1.speak();
24     a1.Write();
25     a1.Author();
26 }
27 }
28 }
```

Author speaks!
Author writes!
I am author!

Task-6

```
1  #include<iostream>
2  using namespace std;
3  class driver{
4  public:
5      void Driver(){
6          cout<<"Driver Drives the Car!"<<endl;
7      }
8  };
9  class mechanic{
10 public:
11     void Mechanic(){
12         cout<<"Mechanic Repairs the Car!"<<endl;
13     }
14 };
15 class technician:public driver,public mechanic{
16 public:
17     void Technician(){
18         cout<<"I can Do both!"<<endl;
19     }
20 };
21 };
22 int main(){
23     technician t1;
24     t1.Driver();
25     t1.Mechanic();
26     t1.Technician();
27 }
```

```
Driver Drives the Car!
Mechanic Repairs the Car!
I can Do both!
```

Task-7

```
Yash
120000
C++
```

```
1  #include<iostream>
2  using namespace std;
3  class Person{
4      public:
5          string name;
6
7  };
8
9  class Employee:public Person{
10     public:
11         float salary;
12 };
13
14 class Developer:public Employee{
15     public:
16         string programmingLan;
17     void DisplayDetails(){
18         cout<<name<<endl;
19         cout<<salary<<endl;
20         cout<<programmingLan<<endl;
21     }
22 };
23
24 int main(){
25     Developer d1;
26     d1.name = "Yash";
27     d1.programmingLan = "C++";
28     d1.salary = 120000;
29     d1.DisplayDetails();
30
31 }
32
```

Task-8

```
1  #include <iostream>
2  using namespace std;
3  class Appliance {
4
5      public:
6          bool power;
7
8
9      Appliance() :power(false){}
10
11     void powerOn() {
12         power = true;
13         cout << "Power ON\n";
14     }
15 };
16 class WashingMachine : public Appliance {
17     public:
18     void wash() {
19         if (power)
20             cout << "Washing clothes...\n";
21         else
22             cout << "Turn power ON first\n";
23     }
24 };
25 class SmartWashingMachine : public WashingMachine {
26     public:
27     void smartControl() {
28         cout << "Smart control activated via mobile app\n";
29         wash();
30     }
31 };
32 int main() {
33     SmartWashingMachine sm;
34     sm.powerOn();
35     sm.smartControl();
36
37 }
```

Power ON
Smart control activated via mobile app
Washing clothes...

Task-9

```
1 #include<iostream>
2 using namespace std;
3 class Account{
4     public:
5         string accountNo;
6
7         void showAccount(){
8             cout<<"Account No: "<<accountNo<<endl;
9         }
10
11 };
12
13 class SavingAccount:public Account{
14     private:
15         float amount;
16         float interestRate;
17     public:
18         SavingAccount(float amount):interestRate(0.4){
19             this ->amount =amount;
20         }
21
22
23         void CalculateInterest(){
24             showAccount();
25             cout<<"Interest: "<<amount*interestRate<<endl;
26         }
27 };
28
```

```
class CurrentAccount:public Account
{
private:
    double balance;
    double overdraftLimit;

public:
    CurrentAccount(double b, double limit)
    {
        balance = b;
        overdraftLimit = limit;
    }

    void withdraw(double amount)
    {
        balance -= amount;
    }

    double calculateOverdraft()
    {
        if (balance < 0)
        {
            return -balance;
        }
        return 0;
    }

    void display()
    {
        showAccount();
        cout << "Current Balance: " << balance << endl;
        cout << "Overdraft Used: " << calculateOverdraft() << endl;
    }
};
```

```
int main(){  
  
    SavingAccount sa1(150000);  
    sa1.accountNo = "PXB990000SBX";  
    sa1.CalculateInterest();  
  
    CurrentAccount ca1(5000,25000);  
    ca1.accountNo="PXB990000SBX";  
    ca1.withdraw(20000);  
    ca1.calculateOverdraft();  
    ca1.display();  
  
}
```

```
Account No: PXB990000SBX  
Interest: 60000  
Account No: PXB990000SBX  
Current Balance: -15000  
Overdraft Used: 15000
```

Task-10

```
--- Book ---  
Title: Killer Hive  
Author: Monet  
  
--- Video ---  
Title: OOP Lecture  
Duration: 60  
Resolution: 1080p  
  
--- Audio ---  
Title: Podcast  
Duration: 45  
Format: MP3
```

```
1  #include<iostream>
2  using namespace std;
3  class Media
4  {
5  protected:
6      string title;
7
8
9  public:
10     Media(string t)
11     {
12         title = t;
13     }
14
15     void displayMedia()
16     {
17         cout << "Title: " << title << endl;
18     }
19
20 };
21
22 class Book : public Media
23 {
24 private:
25     string author;
26
27 public:
28     Book(string t,string a) : Media(t)
29     {
30         author = a;
31     }
32
33     void displayBook()
34     {
35         displayMedia();
36         cout << "Author: " << author << endl;
37     }
38 };
```

```
39 class Video : public Media
40 {
41 private:
42     string resolution;
43     float duration;
44
45 public:
46     Video(string t, float d,string r) : Media(t)
47     {
48         resolution = r;
49         duration =d;
50     }
51
52     void displayVideo()
53     {
54         displayMedia();
55         cout<<"Duration: "<<duration<<endl;
56         cout << "Resolution: " << resolution << endl;
57     }
58 };
59
60 class Audio : public Media
61 {
62 private:
63     string format;
64     float duration;
65
66 public:
67     Audio(string t, float d, string f) : Media(t)
68     {
69         format = f;
70         duration = d;
71     }
72
73     void displayAudio()
74     {
75         displayMedia();
76         cout<<"Duration: "<<duration<<endl;
77         cout << "Format: " << format << endl;
78     }
79 };
```



```
int main()
{
    Book b("Killer Hive", "Monet");
    Video v("OOP Lecture", 60, "1080p");
    Audio a("Podcast", 45, "MP3");

    cout << "\n--- Book ---" << endl;
    b.displayBook();

    cout << "\n--- Video ---" << endl;
    v.displayVideo();

    cout << "\n--- Audio ---" << endl;
    a.displayAudio();
}
```

Task-11

```
1  #include<iostream>
2  using namespace std;
3  class University
4  {
5  protected:
6      string uniName;
7
8  public:
9      University(string name)
10     {
11         uniName = name;
12     }
13
14     void showUniversity()
15     {
16         cout << "University: " << uniName << endl;
17     }
18 };
19 class Faculty : public University
20 {
21 protected:
22     string facultyName;
23
24 public:
25     Faculty(string uName, string fName)
26         : University(uName)
27     {
28         facultyName = fName;
29     }
30
31     void showFaculty()
32     {
33         cout << "Faculty: " << facultyName << endl;
34     }
35 };
```

```
class Administration
{
protected:
    string adminRole;

public:
    Administration(string role)
    {
        adminRole = role;
    }

    void showAdministration()
    {
        cout << "Administration Role: " << adminRole << endl;
    }
};
```

```
class HOD : public Faculty, public Administration
{
private:
    string hodName;

public:
    HOD(string uName, string fName, string role, string hName)
        : Faculty(uName, fName), Administration(role)
    {
        hodName = hName;
    }

    void showHOD()
    {
        cout << "HOD Name: " << hodName << endl;
    }

    void displayAll()
    {
        showUniversity();
        showFaculty();
        showAdministration();
        showHOD();
    }
};

int main()
{
    HOD h("FAST University", "Computer Science", "Academic Head", "Dr. Ahmed");

    h.displayAll();
}
```

```
University: FAST University
Faculty: Computer Science
Administration Role: Academic Head
HOD Name: Dr. Ahmed
```

Task-12

```
1  #include <iostream>
2  using namespace std;
3  class Device
4  {
5  protected:
6      string brand;
7      int powerRating;
8
9  public:
10     Device(string b, int p)
11     {
12         brand = b;
13         powerRating = p;
14     }
15
16     void showDevice()
17     {
18         cout << "Brand: " << brand << endl;
19         cout << "Power Rating: " << powerRating << "W" << endl;
20     }
21 };
```

```
class Computer : public Device
{
protected:
    string processor;
    int ram;

public:
    Computer(string b, int p, string proc, int r)
        : Device(b, p)
    {
        processor = proc;
        ram = r;
    }

    void showComputer()
    {
        cout << "Processor: " << processor << endl;
        cout << "RAM: " << ram << "GB" << endl;
    }
};
```

```
class Network
{
protected:
    string ipAddress;
    int bandwidth;

public:
    Network(string ip, int bw)
    {
        ipAddress = ip;
        bandwidth = bw;
    }

    void showNetwork()
    {
        cout << "IP Address: " << ipAddress << endl;
        cout << "Bandwidth: " << bandwidth << " Mbps" << endl;
    }
};
```

```
class Server : public Computer, public Network
{
private:
    string serverType;

public:
    Server(string b, int p, string proc, int r,
           string ip, int bw, string type)
        : Computer(b, p, proc, r),
          Network(ip, bw)
    {
        serverType = type;
    }

    void showServer()
    {
        cout << "Server Type: " << serverType << endl;
    }

    void displayAll()
    {
        showDevice();
        showComputer();
        showNetwork();
        showServer();
    }
};
```

```
int main()
{
    Server s("Dell", 750, "Intel i7 ", 64,
            "192.168.1.10", 1000, "Web Server");

    s.displayAll();
}
```

```
Brand: Dell
Power Rating: 750W
Processor: Intel i7
RAM: 64GB
IP Address: 192.168.1.10
Bandwidth: 1000 Mbps
Server Type: Web Server
```

Task-13

```
#include <iostream>
using namespace std;
class Base
{
public:
    int publicVar = 10;

protected:
    int protectedVar = 20;

private:
    int privateVar = 30;
};
class PublicDerived : public Base
{
public:
    void show()
    {
        cout << "PublicDerived accessing publicVar: " << publicVar << endl;
        cout << "PublicDerived accessing protectedVar: " << protectedVar << endl;
        // cout << privateVar; // Not accessible
    }
};
```

```
class ProtectedDerived : protected Base
{
public:
    void show()
    {
        cout << "ProtectedDerived accessing publicVar: " << publicVar << endl;
        cout << "ProtectedDerived accessing protectedVar: " << protectedVar << endl;
        // cout << privateVar; // Not accessible
    }
};
class PrivateDerived : private Base
{
public:
    void show()
    {
        cout << "PrivateDerived accessing publicVar: " << publicVar << endl;
        cout << "PrivateDerived accessing protectedVar: " << protectedVar << endl;
        // cout << privateVar; // Not accessible
    }
};
```

```
int main()
{
    PublicDerived pd;
    pd.show();
    cout << "Accessing publicVar from main through PublicDerived: " << pd.publicVar << endl;
    // cout << pd.protectedVar; // Not accessible
    // cout << pd.privateVar; // Not accessible

    ProtectedDerived protd;
    protd.show();
    // cout << protd.publicVar; // Not accessible |

    PrivateDerived privd;
    privd.show();
    // cout << privd.publicVar; // Not accessible
}
```

```
PublicDerived accessing publicVar: 10
PublicDerived accessing protectedVar: 20
Accessing publicVar from main through PublicDerived: 10
ProtectedDerived accessing publicVar: 10
ProtectedDerived accessing protectedVar: 20
PrivateDerived accessing publicVar: 10
PrivateDerived accessing protectedVar: 20
```

```
1  #include<iostream>
2  using namespace std;
3  class BankAccount
4  {
5  public:
6      int balance;
7
8      void showBalance()
9      {
10         cout << "Balance: " << balance << endl;
11     }
12 };
13 class SavingsAccount : private BankAccount
14 {
15 public:
16     void setBalance(int b)
17     {
18         balance = b;
19     }
20
21     void display()
22     {
23         showBalance();
24     }
25 };
26 int main()
27 {
28     SavingsAccount s;
29
30     s.setBalance(5000);
31     s.display();
32
33     // s.balance = 5000;      // Not accessible
34     // s.showBalance();      // Not accessible
35 }
```

Balance: 5000

Task-15

```
1  #include<iostream>
2  using namespace std;
3  class Person {
4
5  public:
6      void show() {
7          cout << "Person" << endl;
8      }
9  };
10 class Student : public Person {};
11 class Employee : public Person {};
12 class TeachingAssistant : public Student, public Employee{
13
14 };
15 /* error: Both base classes of TA have same show() method.
16    So, Compiler gets Dconfused while calling*/
17 int main(){
18     TeachingAssistant obj;
19     obj.Student::show();
20
21 }
```

Person

Task-16

NO OUTPUT

```
1  #include<iostream>
2  using namespace std;
3  class Machine {
4  public:
5      void start() {
6          cout << "Machine Started" << endl;
7      }
8  };
9  class Printer : public Machine {
10 };
11 class Scanner : public Machine {
12 };
13 class AllInOne : public Printer, public Scanner {
14 };
15 int main() {
16     AllInOne obj;
17
18     obj.start();    // Ambiguous call
19 }
```

Task-17

```
1  #include <iostream>
2  using namespace std;
3  class Machine {
4  public:
5      void start() {
6          cout << "Machine Started" << endl;
7      }
8  };
9  class Printer : virtual public Machine {
10 };
11 class Scanner : virtual public Machine {
12 };
13 class AllInOne : public Printer, public Scanner {
14 };
15 int main() {
16     AllInOne obj;
17     obj.start();    // No ambiguity
18     Machine * m1= (Printer*)&obj;
19     Machine * m2 = (Scanner*)&obj;
20     cout << "Address via Printer path: " << m1 << endl;
21     cout << "Address via Scanner path: " << m2 << endl;
22 }
```



```
Machine Started
Address via Printer path: 0x6ffe30
Address via Scanner path: 0x6ffe30
```

Task-18

```
1  #include <iostream>
2  using namespace std;
3  class Device {
4  public:
5      void powerOn() {
6          cout << "Device Power On" << endl;
7      }
8  };
9  class Camera : virtual public Device {
10 public:
11     void takePhoto() {
12         cout << "Photo Captured" << endl;
13     }
14 };
15 class Phone : virtual public Device {
16 public:
17     void makeCall() {
18         cout << "Calling..." << endl;
19     }
20 };
21 class SmartPhone : public Camera, public Phone {
22 public:
23     void useApps() {
24         cout << "Using Smartphone Apps" << endl;
25     }
26 };
27 int main() {
28     SmartPhone obj;
29     obj.powerOn();
30     obj.takePhoto();
31     obj.makeCall();
32     obj.useApps();
33 }
```

```
Device Power On
Photo Captured
Calling...
Using Smartphone Apps
```

Task-19

```
1  #include <iostream>
2  using namespace std;
3  class Person
4  {
5  protected:
6      string name;
7
8  public:
9      Person(string n = "Unknown")
10     {
11         name = n;
12     }
13
14     void displayPerson()
15     {
16         cout << "Name: " << name << endl;
17     }
18 };
19 class Teacher : virtual public Person
20 {
21 public:
22     Teacher(string n="") : Person(n) {}
23
24     void teachCourse()
25     {
26         cout << name << " is teaching a course" << endl;
27     }
28 };
```

```
class Admin : virtual public Person
{
public:
    Admin(string n="") : Person(n) {}

    void manageSchool()
    {
        cout << name << " is managing school records" << endl;
    }
};

class HeadTeacher : protected Teacher
{
public:
    HeadTeacher(string n) : Person(n), Teacher(n) {}

    void manageTeachers()
    {
        cout << name << " is managing all teachers" << endl;
    }
};

class Coordinator : public Teacher, public Admin
{
public:
    Coordinator(string n) : Person(n), Teacher(n), Admin(n) {}

    void organizeEvent()
    {
        cout << name << " is organizing school event" << endl;
    }
};
```

```
//
class Student : public Person
{
public:
    Student(string n) : Person(n) {}

    void enrollCourse()
    {
        cout << name << " enrolled in a course" << endl;
    }
};

class OfficeAdmin : private Person
{
public:
    OfficeAdmin(string n) : Person(n) {}

    void showAdmin()
    {
        displayPerson();
        cout << "Handling office administration" << endl;
    }
};
```

```
int main()
{
    cout << "---- Student ----" << endl;
    Student s("Ali");
    s.displayPerson();
    s.enrollCourse();

    cout << "\n---- Coordinator ----" << endl;
    Coordinator c("Mr Ahmed");
    c.displayPerson();
    c.teachCourse();
    c.manageSchool();
    c.organizeEvent();

    cout << "\n---- Head Teacher ----" << endl;
    HeadTeacher h("Mrs Sara");
    h.manageTeachers();

    cout << "\n---- Office Admin ----" << endl;
    OfficeAdmin a("Mr Khan");
    a.showAdmin();
}
```

---- Student ----

Name: Ali

Ali enrolled in a course

---- Coordinator ----

Name: Mr Ahmed

Mr Ahmed is teaching a course

Mr Ahmed is managing school records

Mr Ahmed is organizing school event

---- Head Teacher ----

Mrs Sara is managing all teachers

---- Office Admin ----

Name: Mr Khan

Handling office administration